

DeepInterpolation Ephys — Numerical Experiment Report

Abstract

DeepInterpolation is a self-supervised denoiser for extracellular electrophysiology: it learns to predict each voltage sample from its neighbours in time and across the probe, and is never shown a spike. It reliably raises signal-to-noise — but on this benchmark it *lowers* the detectability of the very spikes a downstream spike sorter must find, because cutting white noise leaves behind a smoother, spike-shaped residual that a matched filter cannot separate as cleanly. This report asks which design choice recovers that lost detectability. We trained **49 model variants** — sweeping the loss function, network capacity, output-head width, spike-aware losses, and a family of architectural changes borrowed from the SUPPORT denoiser — and scored each on a **frozen hybrid ground-truth benchmark** with a fast surrogate for spike-sorting performance. Because training is stochastic and the validation loss is nearly flat, we first measured a run-to-run **noise floor** (4–5 seeds per key model) and treat only differences larger than ~ 2 sigma as real. The result: almost every knob we turned is within that noise; extra capacity helps a little (+0.06 d’); but one architectural change dominates — **letting the temporal network see the two frames immediately adjacent to each target sample, instead of hiding them, recovers about half of the lost detectability (+0.20 d’) and simultaneously fixes a long-standing amplitude undershoot**. Two large-scale training runs ($\sim 7\times$ more training) confirm this holds at scale, and show that both metrics saturate within the first ~ 10 thousand updates — so the short sweep runs were *not* undertrained: the ceiling is architectural, not a training-budget limit.

*Scope: this is a numerical architecture/loss study scored by a fast surrogate + template diagnostics, not by a full spike-sorting pipeline. All models are the **fold** DeepInterpolation ephys denoiser (defined below).*

1. Background: the model, the task, and the puzzle

1.1 Self-supervised denoising with a blind spot

DeepInterpolation denoises a recording by learning the *predictable* part of each sample and discarding the rest as noise. For one target sample — a single channel at a single time — the network is shown a window of neighbouring samples (nearby time frames, across all channels) and asked to predict the target. The essential trick is a **blind spot**: the target channel’s own value at the target time is withheld from the input. Since the network can never see the target’s own noise, it cannot copy it; the best it can do is predict the component the target shares with its neighbours, so its output is a denoised estimate. No clean traces and no spike labels are ever needed — this is the Noise2Self / blind-spot principle applied to ephys.

1.2 The model under study (“the champion”)

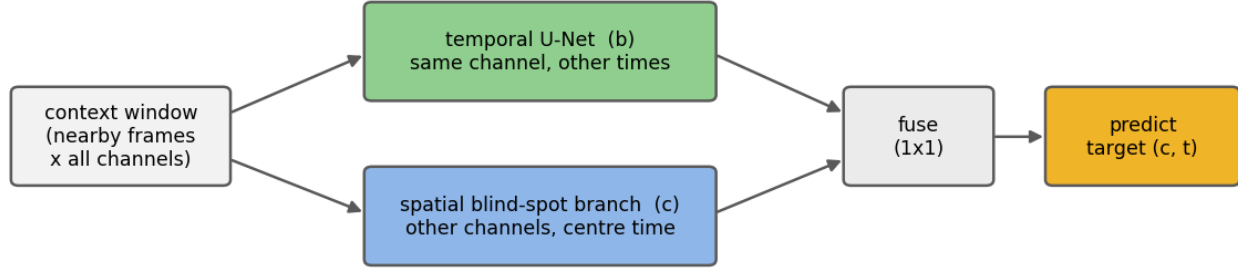
Our reference model — the **champion** — predicts each target from two branches whose outputs are fused per channel (Figure 1a):

- a **temporal branch** (Figure 1b): a 1-D U-Net that reads a window of neighbouring *time frames* (here ± 30) on the target channel and its neighbours, and predicts the target from how the signal evolves in time;
- a **spatial blind-spot branch** (Figure 1c): a small dilated-convolution network that reads the *same-time* values on the *other* channels (the target channel excluded) and predicts the target from the probe’s spatial structure.

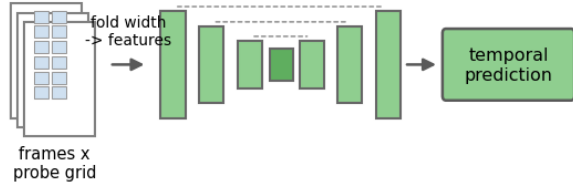
Three design choices recur throughout the report:

- **The omission gap.** To stop the temporal branch from trivially copying adjacent samples (which are the most likely to share noise), the champion hides not only the target frame t but also its immediate neighbours $t\pm 1$, feeding only frames $t\pm 2$ outward to the temporal branch. The number of hidden neighbours is the **omission** setting: **omission=1** (champion) hides $t\pm 1$; **omission=0** hides nothing but t , so the temporal branch gets to see $t\pm 1$. This single choice turns out to be the crux of the study (Figure 2; Section 4.7).
- **Geometry (“fold”).** Neuropixels channels sit on a 2-D grid. Instead of an expensive 2-D convolution, the champion *folds* the short (width) axis into feature channels and runs a cheap 1-D U-Net along the long (depth) axis — matching 2-D denoising quality at ~ 1 -D cost. Its blind spot excludes the target’s whole depth-row.
- **The knobs we sweep:** **capacity** (U-Net width, **base_channels**), the width of the pointwise **fuse head** that merges the two branches, the **loss function**, **spike-aware loss** weightings, and several **SUPPORT-style wiring** options (Section 4.5). The champion is 0.85 M parameters.

a Two branches predict each sample; the target sample itself is held out (the blind spot).

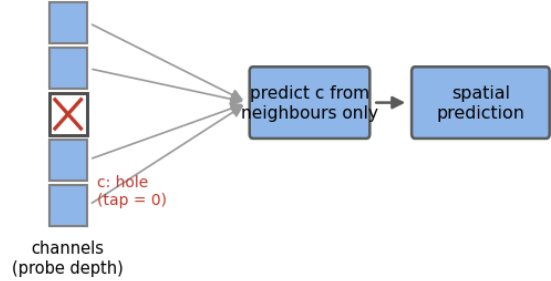


b Temporal branch: a 1-D U-Net along probe depth



Reads the SAME channel at neighbouring times ($t \pm 2..t \pm 31$; the omission gap hides $t \pm 1$ — Figure 2). Channels are scattered onto the probe grid and the narrow width axis is folded into feature maps, so a cheap 1-D U-Net runs along depth (encode / pool $\rightarrow$ bottleneck $\rightarrow$ decode, with skip connections).

c Spatial blind-spot branch: hole convolution



Reads OTHER channels at the centre frame to predict c , with c 's own value excluded by a centre-zeroed ("hole") kernel. A stack of dilated hole-convs (dilations 1,2,4,8,16) widens spatial reach while preserving the hole — so the prediction of c never depends on c itself.

Figure 1 — The champion denoiser. (a) Each output sample is predicted by two branches — a **temporal U-Net** (reads the same channel at neighbouring times) and a **spatial blind-spot branch** (reads the other channels at the centre time) — combined by a 1×1 fuse head; the target sample itself is never an input (the blind spot). (b) The temporal branch scatters the channels onto the probe grid, folds the narrow width axis into feature maps, and runs a cheap 1-D U-Net (encode–bottleneck–decode with skips) along the long probe-depth axis. (c) The spatial branch is a stack of **hole convolutions**: the kernel’s centre tap is zeroed, so each channel is predicted only from its neighbours, never from its own value — the mechanism that makes the model blind to the target.

1.3 The puzzle: denoising helps SNR but hurts detection

The champion does its nominal job — it raises peak signal-to-noise from **5.8** to **~7.7**. But the number that matters for spike sorting moves the *wrong way*. We measure **detectability** as how separable a unit’s spikes are from the background under a matched filter (a d' , defined in Section 2). Raw data scores $d' = 4.50$; every denoised model sits *below* it, at 4.0–4.3. So denoising makes spikes **harder** to detect, not easier. The mechanism (Section 4.2) is that removing white noise leaves a correlated, spike-shaped residual in the background that the matched filter mistakes for signal. **Recovering that lost detectability — pushing d' back toward 4.50 — is the optimization target of the entire study.**

2. Methods: benchmark, metrics, and the noise floor

2.1 Frozen hybrid benchmark

Every model is scored on one fixed benchmark: a real Neuropixels 1.0 recording (ecephys_681532, ProbeC) into which 10 ground-truth units were injected at known times and amplitudes — a *hybrid* recording. Because the true spike times and waveforms are known, we measure detection and waveform fidelity directly, without running a spike sorter (the surrogate below stands in for one). The evaluation is identical for all models — same 10 units, same seed=0 subsample of 100 spikes/unit + 200 background windows — so **every difference between models reflects training, not the eval.**

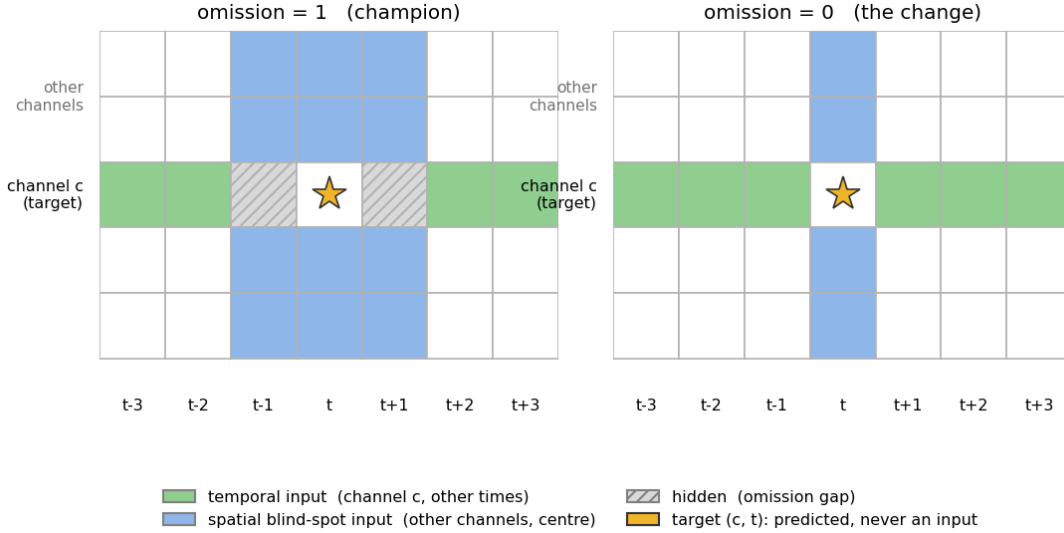


Figure 2 — The one architectural change this study turns on: the omission gap. Which samples each branch reads to predict the target (star). At **omission=1** (champion, left) the frames immediately flanking the target — $t\pm 1$ — are hidden from the temporal branch (hatched); the model must reach out to $t\pm 2$ and beyond, and relies on the spatial branch to pick up $t\pm 1$ on the *other* channels. At **omission=0** (right) the temporal branch sees $t\pm 1$ directly and the spatial branch reads only the centre frame. Section 4.7 shows this single change is the largest lever in the sweep.

2.2 The two metrics

We report two numbers that measure genuinely different things:

- **d' (detectability) — the sorting-relevant metric.** For each unit we form a template and slide it over the trace as a matched filter; d' is the standardized separation between the filter's response at true spike times versus background (higher = easier to detect). We compute it two ways: d'_{self} uses a template built from the *denoised* data (what a sorter operating on denoised traces sees), and d'_{fixed} uses the *true/raw* template (a control: if d'_{self} rises but d'_{fixed} does not, the gain is a self-consistent artifact, not real signal). The raw-data reference is $d' = 4.50$, and every denoised model falls below it, so *higher $d' = \text{less damage}$* .
- **amp_ratio (peak fidelity).** The denoised spike's peak-to-peak amplitude divided by the true template's, on the peak channel: 1.0 means the peak is preserved, < 1.0 means the denoiser flattened it. This is a fidelity measure and, as Section 5 shows, is only loosely coupled to detectability.

Supporting diagnostics (waveform-shape correlation `temporal_cos/spatial_cos`, trough width `fwhm_ratio`, and SNR) are quoted where they pin down a mechanism.

Champion baseline (the reference for every delta): fold geometry, `base_channels=32`, `depth=3`, temporal branch omitting 3 centre frames (`omission=1`), spatial branch `bs_channels=64/bs_depth=5`, `fuse_channels=64`, Charbonnier loss — **0.85 M parameters**; it scores d'_{self} 4.16, amp 0.845 (seed means, below).

2.3 The noise floor: why single runs cannot be trusted

Two facts make any *single* training run an unreliable ranking. (i) Training is stochastic — GPU non-determinism, random initialisation and data order. (ii) The validation loss is nearly flat: spikes are only 0.065% of samples, so they barely move a sample-averaged loss (Section 2.4 measures exactly how little), and the “best” checkpoint is effectively picked at random along a plateau. We therefore retrained the key configurations **4–5 times, changing only the seed**, and measured how far the metrics wander on their own:

config	amp_ratio (mean +/- sd)	d'_{self} (mean +/- sd)
champion (base32)	0.845 +/- 0.009	4.164 +/- 0.028
fuse256	0.844 +/- 0.014	4.201 +/- 0.037
base64	0.851 +/- 0.015	4.228 +/- 0.038
champ_l2 (n=4)	0.849 +/- 0.012	4.221 +/- 0.064

The spread is **sigma** ~ 0.01 in amp and **0.03–0.06** in d' , which fixes the **decision rule for the rest of the report: a single-run difference is real only if it exceeds $\sim 2\text{sigma}$ (~ 0.02 amp, ~ 0.07 d')**. This one measurement is what

keeps the sweep honest. Several apparent “wins” evaporate under it: by a Welch t-test against the champion, **base64** is **+0.063 d’ (p=0.019, real)**, but fuse256 (+0.036, p=0.12) and even L2 (+0.057, p=0.18) are not — their headline single runs (fuse256 seed 0 = 4.238, champ_l2 seed 0 = 4.296) were high draws, just as the champion’s own seed 0 (4.151) was a low draw that had inflated every early comparison. **Read every ranking below against this +/-2sigma band.**

2.4 A direct measurement: how much can spikes even move the loss?

The natural metric would be the model’s own objective — the held-out validation loss — so it is worth showing *concretely* why we do not rank models with it. That loss is a per-channel MSE between the blind-spot prediction and the noisy target, averaged over **every** channel and sample; spikes are a vanishingly small fraction of those samples. Using the known GT spike times, we split all (channel, sample) points into spike-support points (on a spike’s channels, near its time) and background, and measured how far the loss would fall if the network reconstructed **every spike perfectly** — i.e. if the residual at spike-support points dropped to the local off-spike floor:

quantity	champion	fuse256
p_spike — fraction of points that are spike-support	0.065%	0.065%
mean squared residual at spike-support points	0.528	0.511
local off-spike floor on the same channels	0.463	0.463
loss drop if every spike were reconstructed perfectly (dL)	4.2e-5	3.2e-5

The result is decisive. Spike-support points are only **0.065%** of the data, and even *at* a spike only ~12% of the residual is recoverable spike error (the rest is the same noise floor as the surrounding background). Multiplying through, reconstructing every spike **perfectly** would lower the validation loss by just **~4e-5** — **about 0.007%** of its ~0.57 value. That loss already varies by **~7.6e-4 between training seeds**, so the *entire achievable spike contribution is ~18x smaller than the loss’s own run-to-run noise*.

This is the concrete reason the validation loss cannot rank denoisers on spike quality — spike reconstruction is a rounding error in it. It is why the “best” checkpoint is essentially a random draw along a flat plateau (Section 2.3), and why every model in this report is scored with the spike-level **d’ surrogate** instead.

The same measurement also argues for long training. Because spikes are only 0.065% of the data, the gradient that teaches the network to reconstruct them is a correspondingly tiny share of each update — it accumulates very slowly, and a single pass over the data delivers almost none of it. Shaping faithful spike reconstruction from so faint a signal plausibly requires revisiting spikes many times over, i.e. training far longer than the one epoch used throughout this sweep. That is exactly the regime the **SUPPORT** denoiser adopts — roughly 500 epochs (~14M gradient updates) on a single recording, about 28x our budget (Section 7). So the measurement does double duty: the property that blinds the validation loss to spikes (forcing us onto the surrogate) *also* predicts that spike-level detectability may keep improving long after that loss has gone flat — the undertraining hypothesis we test at SUPPORT scale in Section 7.

3. Master results table (all scored models, sorted by d’_self)

The table below is the full ledger — every checkpoint we scored, sorted by d’_self and read against the +/-2sigma noise band from Section 2 (a row matters only if it clears the band). Two rows do. **base64** clears it modestly (+0.063, the reproducible capacity gain); and far above everything else sits **omission0** — the omission-gap change of Section 4.7 — at d’ **4.368**, well past the band and topping the amplitude column too. Everything in between (L2, fuse-width, spike-weighting, SUPPORT wiring) falls inside the band and is treated as noise. Figure 3 shows the same values with the champion’s band shaded; the four configurations we retrained across seeds carry +/-2sigma error bars, the rest are single runs.

d’ columns share raw baseline 4.497. amp baseline (champion mean) = 0.845 +/- 0.009.

model	family	params	amp	d’_self	d’_fixed	fwhm	snr_deep
omission0 (seed0)	temporal: t+/-1 visible	0.85M	0.939	4.368	4.396	0.976	7.01
champ_l2 (seed0)	loss L2, base32	0.85M	0.858	4.296	4.326	1.020	7.55
base64 (seed0)	capacity	3.15M	0.877	4.273	4.283	1.003	7.86
archL3	15x + weight lambda3	12.59M	1.076	4.256	4.285	1.024	7.62

model	family	params	amp	d'_self	d'_fixed	fwhm	snr_deep
fuse256_uL100_last	fuse256 + ubias lambda100	0.93M	0.858	4.250	4.280	1.048	7.58
fuse256_l2	L2 + fuse256	0.93M	0.853	4.249	4.281	1.062	—
fuse256_uL100	fuse256 + ubias lambda100	0.93M	0.856	4.242	4.282	1.062	7.52
support_all_l2	L2 + SUPPORT all	0.90M	0.847	4.241	4.265	1.020	—
base64_l2	loss L2 + capacity	3.15M	0.867	4.239	4.268	1.011	7.60
fuse256 (seed0)	fuse width	0.93M	0.841	4.238	4.264	1.014	7.68
support_sd_l2	L2 + SUPPORT sd	0.85M	0.830	4.228	4.262	1.006	—
fuse512_wL3	fuse512 + weight lambda3	1.14M	0.871	4.223	4.262	1.029	7.46
weighted (lambda3)	spike weight	0.85M	0.845	4.219	4.264	1.014	7.64
support_sd	SUPPORT stage+dense	0.85M	0.852	4.209	4.246	1.003	7.70
fuse256_tmult8	temporal_mult=8	0.93M	0.874	4.203	4.242	1.031	7.61
base64_l2_last	L2 + capacity (last)	3.15M	0.856	4.203	4.239	1.020	7.41
no_norm (seed0)	no GroupNorm	0.85M	0.849	4.200	4.232	0.989	7.65
fuse256_uL300	fuse256 + ubias lambda300	0.93M	0.858	4.182	4.204	1.020	7.28
archL10	15× + weight lambda10	12.59M	1.474	4.181	4.225	0.991	7.53
fuse256_uL30	fuse256 + ubias lambda30	0.93M	0.850	4.181	4.209	1.031	7.58
tmult8	temporal_mult=8 (base)	0.85M	0.827	4.176	4.212	0.992	7.51
ho (bs_frames=1)	blind-spot width	0.85M	—	4.176	4.211	—	7.56
l10g1	weight lambda10	0.85M	0.859	4.172	4.210	1.014	7.52
champ_uL100	gamma1 body + ubias lambda100	0.85M	0.854	4.164	4.209	1.009	7.47
champ (seed0)	baseline	0.85M	0.834	4.151	4.192	1.031	7.69
l10g2	weight lambda10 gamma2	0.85M	0.859	4.148	4.186	1.034	7.47
fuse1024	fuse width	1.97M	0.831	4.147	4.186	1.037	7.48
fuse256_hard1000	hard gate lambda1000	0.93M	0.857	4.147	4.188	0.992	7.17
fuse128	fuse width	0.87M	0.853	4.145	4.168	0.994	7.59
fuse512	fuse width	1.14M	0.841	4.142	4.183	0.975	7.54
support_all	SUPPORT +multiscale	0.90M	0.841	4.099	4.144	1.020	7.66
fuse256_wL3	fuse256 + weight lambda3	0.93M	0.825	4.063	4.108	1.045	7.30
arch	15× capacity (plain)	12.59M	0.828	3.958	4.025	1.099	8.61

(*_last* checkpoints and the replicate seeds omitted here; replicates summarized in Sec. 2. The **champ_l2** row is its **seed-0** run — its 4-seed mean is 4.221 (Sec. 2, Sec. 4.1). amp for **support*_last**: support_sd_last 0.848, support_all_last 0.846; L2-combo **_last**: support_sd_l2_last d' 4.243 / amp 0.858, support_all_l2_last 4.241 / 0.860.)

4. The design sweep — experiment by experiment

4.1 Loss shape × capacity (2×2)

The champion trains with a Charbonnier loss — a smooth blend of L1 and L2 that is deliberately *robust to outliers*. That sounds desirable until you remember that a spike **is** an outlier: the robust loss shrugs off the large error at the spike peak, so the network is never strongly pushed to reconstruct it. Plain L2 (mean-squared error) removes that leniency — large errors are punished quadratically. We tested this at two capacities, giving a clean 2×2 (Figure 4):

	Charbonnier	L2 (MSE)
base32 (0.85M)	4.164	4.296 *
base64 (3.15M)	4.228	4.239

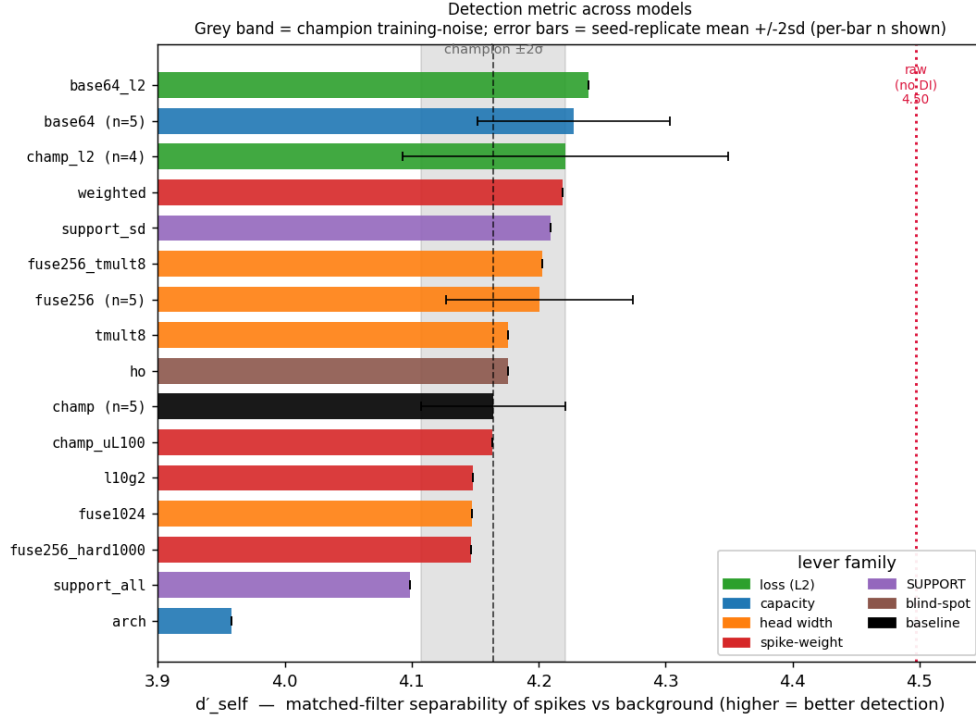


Figure 3 — d' across all models vs the noise band. Bars are d'_self (higher = better detection); the grey stripe is the champion $\pm 2\sigma$ (5 seeds); the dotted line is the raw data (4.50 — every model sits below it). `champ/fuse256/base64` (5-seed) and `champ_l2` (4-seed) are shown as mean $\pm 2\sigma$ error bars; all others are single runs. After replication **only base64 clears the band**; both `champ_l2` (mean 4.221) and `fuse256` have error bars overlapping it — their high single runs were lucky seeds — and spike-weighting, SUPPORT wiring, head width and blind-spot width are all noise.

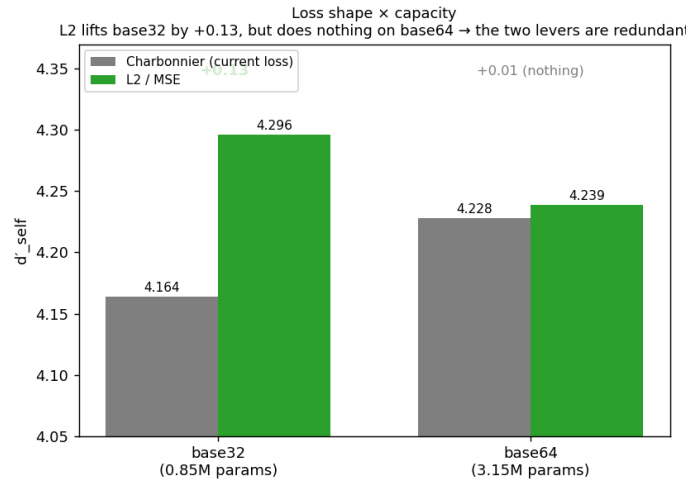


Figure 4 — loss $\times$ capacity (seed-0 runs). At seed 0, L2 lifts base32 by +0.13 and adds nothing on base64. That +0.13 is a lucky seed, though: the 4-seed `base32+L2` mean is 4.221 (+0.057, NS — Sec. 4.1). The robust takeaway is the *redundancy* — L2 and capacity clean the same residual, so they don't stack.

- On the champion (base32), seed 0 shows a striking **+0.132** d' (4.16→4.30), corroborated by d'_fixed (+0.134) and a slight SNR *drop* (7.69→7.55) — a real separability gain, not peak inflation. On base64 it adds essentially nothing (+0.011).

But that +0.13 was a high draw. Replicated over 4 seeds (Section 2), **base32+L2** averages **4.221 +/- 0.064** — **+0.057 vs champion, p=0.18 (not significant)**; amplitude is flat too (+0.004). L2 is a *small, free* nudge, not the ~4sigma win the single run implied.

Does anything stack on L2? Combining it with the SUPPORT wiring or the fat fuse head does not help:

L2 combo	d'_self	amp
champ_l2 — 4-seed mean	4.221 +/- 0.064	0.849
support_sd_l2	4.228	0.830
support_all_l2	4.241	0.847
fuse256_l2	4.249	0.853

All three land **inside champ_l2's own seed spread (4.155–4.296)** — above its mean but *below* its seed-0 run (their matched seed) — so none clears the noise band. L2 **subsumes** capacity and wiring rather than stacking with them (base64_l2 = 4.239 tells the same story). **Net: base32 + L2 stays the efficient pick, but the L2 gain is small (+0.057) and not statistically established on this substrate.**

4.2 Capacity — and why more denoising is not better detection

The obvious lever is size: a bigger network should denoise better. It does — but “denoises better” (higher SNR) turns out to be nearly the *opposite* of “detects better” (higher d'). The 15× **arch** model removes the most noise of anything we trained and is simultaneously the **worst** detector, because the extra capacity is spent sculpting the leftover background into smooth, spike-like structure that fools the matched filter. Figure 5 shows the dissociation directly.

model	params	d'_self	delta vs champ mean (4.164)
base64 (replicate mean)	3.15M	4.228 +/- 0.038	+0.063 (p=0.019)
arch (15×, plain)	12.59M	3.958	-0.206 (worst)

- base64 = the only *reproducible* architectural d' gain (+0.06), but 3.15M params.
- **arch (15×) is the worst d' (3.958) yet the highest SNR (8.61 / dsnr 2.81)** — the clearest SNR vs d' dissociation: raw capacity maximizes SNR while injecting the most correlated residual. Capacity != separability.

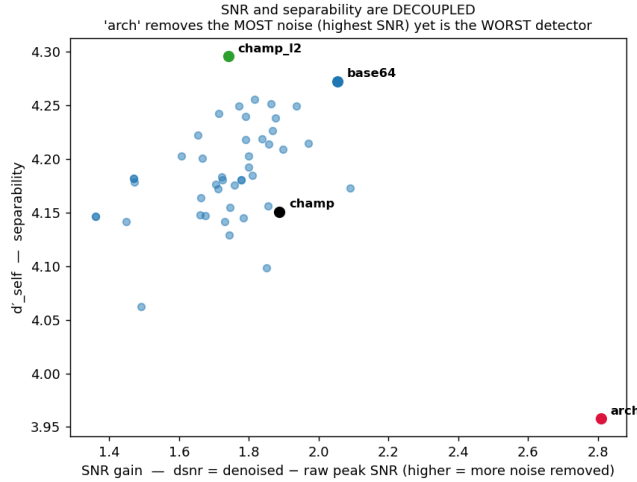


Figure 5 — the SNR trap. SNR gain (x) vs separability (y). If denoising helped detection these would trend up-and-right; instead it is flat-to-negative. **arch** removes the MOST noise (highest SNR) yet is the WORST detector. **champ_12** (seed-0) posts the top d' with a below-average SNR gain — SNR is a misleading target for sorting.

4.3 Fuse-decoder width (single runs)

fuse128 4.145 ; fuse256 4.238 ; fuse512 4.142 ; fuse1024 4.147. **Non-monotonic** (only 256 breaks out) → fuse256's apparent gain is a seed draw (confirmed by Sec. 2: fuse256 mean 4.201, +0.036 NS). temporal_mult=8: tmult8 4.176, fuse256_tmult8 4.203 — no lever.

4.4 Spike-weighting (peak-aware loss)

- Legacy magnitude weight: weighted(lambda3) 4.219, l10g1 4.148–4.172, l10g2 4.148; **amp rises with lambda** (0.834→0.859) but **d' flat/noisy**. At lambda10 on 15× (archL10) amp **over-shoots to 1.474**.
- Unbiased position gate: champ_uL100 4.164, fuse256_uL30/100/300 = 4.18–4.24; **no dose-response on d'**.
- Hard binary gate (62% spike loss-share): fuse256_hard1000 4.147 = champion; **loss curve moved but d' didn't** → **headroom ceiling confirmed**. Spike-weighting fixes amplitude, not detection.

4.5 SUPPORT wiring restore

SUPPORT is a published blind-spot denoiser; our **fold** champion is, in effect, a simplified version of it that dropped three pieces of wiring (dense re-injection of the centre input, staging the temporal feature into the blind branch, and a parallel multi-scale stack). A natural hypothesis was that those simplifications caused the amplitude undershoot, so we put them back:

variant	params	amp (best)	d'_self
support_sd (stage+dense)	0.85M	0.852	4.209
support_all (+multiscale)	0.90M	0.841	4.099 (worse)

- stage+dense: small nudge, within noise vs champion mean; multiscale **hurts d' and overfits hardest**. Restoring the wiring did *not* fix anything — the simplifications were not the cause of the undershoot.

The multiscale variant also exposed a training pathology worth recording (Figure 6): the SUPPORT variants carry enough extra plumbing to **overfit the background within a single epoch**, reaching their best validation loss at ~7% of training and then drifting back up, whereas the champion family never overfits in one pass. Since that validation loss is exactly what selects the saved checkpoint, the extra wiring quietly makes checkpoint selection *worse*.

4.6 Blind-spot width

bs_frames 3→1 (ho): d' 4.176 vs champion 4.151–4.164 (delta+0.025, NS). Not a lever.

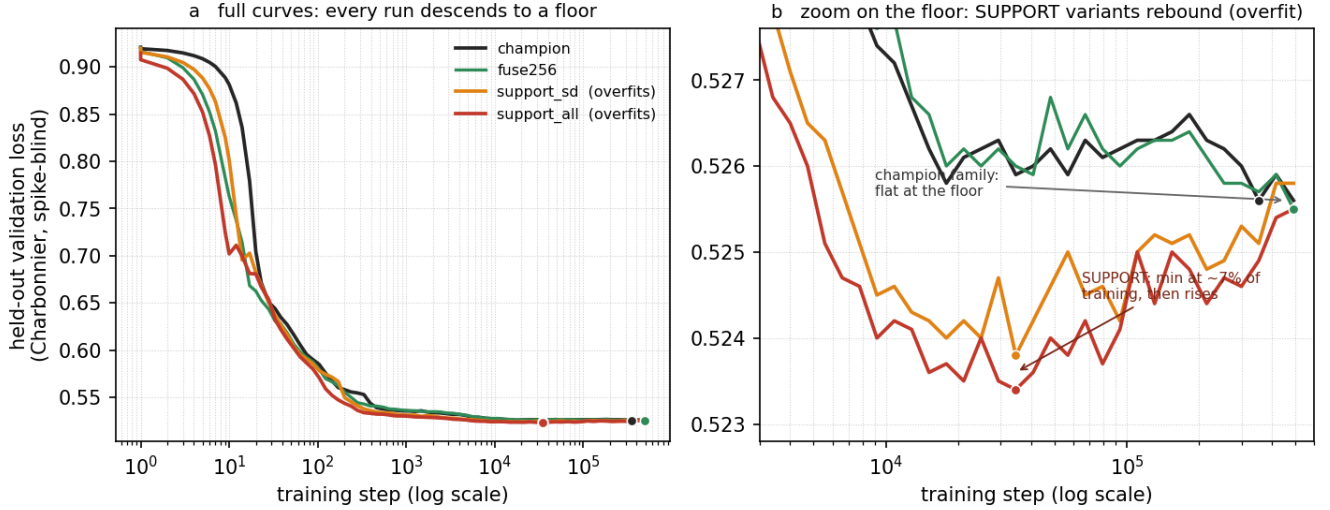


Figure 6 — SUPPORT variants overfit within one epoch; the champion family does not. Held-out validation loss (Charbonnier; spike-blind, Section 2.3) vs training step, for two champion-family runs (champion, fuse256) and the two SUPPORT-wiring variants, all trained identically. (a) Full curves: every run descends from ~0.92 to a floor near 0.525. (b) Zoom on that floor: the SUPPORT variants reach their minimum at only ~7% of training (dots) then drift **back up** by ~0.002 — overfitting the background — while champion and fuse256 keep descending and stay at the floor. Because the saved checkpoint is chosen at this minimum, the extra wiring quietly makes checkpoint selection worse.

4.7 The omission gap — recovering $t \pm 1$ (the biggest single lever)

Every experiment so far kept one champion choice fixed: the **temporal omission gap** (omission=1, Figure 2), which hides the frames immediately flanking the target ($t \pm 1$) from the temporal U-Net, forcing it to interpolate the sharp spike peak from frames $t \pm 2 \dots t \pm 31$. SUPPORT makes the opposite choice — it blinds only the target frame and lets its temporal branch see $t \pm 1$. We tested that directly (omission=0, which also forces the blind spot down to 1 centre frame), alongside a second SUPPORT divergence — dropping GroupNorm (no_norm):

run	amp_ratio	fwhm	d'_self	d'_fixed	SNR gain
champion (omission=1, seed0)	0.834	1.031	4.151	4.192	+1.89
no_norm (no GroupNorm)	0.849	0.989	4.200	4.232	+1.84
omission0 ($t \pm 1$ visible)	0.939	0.976	4.368	4.396	+1.20

omission0 is the **largest single move in the entire sweep, on both axes at once** — and, uniquely, it moves *amplitude*, which had been architecture-invariant at 0.845 ± 0.01 across capacity, fuse width, SUPPORT wiring and L2. Peak attenuation drops from ~15% to ~6% (amp $0.834 \rightarrow 0.939$), the temporal broadening disappears (fwhm $1.031 \rightarrow 0.976$), and detection rises $+0.204 d'$ (~7sigma vs the champion band) on **both** self- and fixed-template scores — a genuine shape gain, not self-template inflation. no_norm does essentially nothing ($+0.036 d'$, $+0.015$ amp, both within noise): the lever is specifically the omission design, not the absence of normalization.

The per-unit table is where the mechanism is clearest. Recall (Section 5) that the champion's amplitude undershoot lives almost entirely on the **weak, marginal units** — the blind spot rebuilds their peak from noisy neighbours and hedges toward baseline. Giving the temporal branch $t \pm 1$ back rescues *exactly those units*:

unit	raw d'	champ amp	no_norm amp	omission0 amp	champ d'	no_norm d'	omission0 d'
2143	12.53	0.968	0.976	1.002	12.37	12.28	12.57
793	8.57	0.966	0.972	0.956	8.37	8.47	8.43
1143	5.32	0.881	0.897	0.946	4.84	4.95	5.14
1300	4.07	0.857	0.859	0.923	3.61	3.65	3.88
1122	3.80	0.854	0.882	0.944	3.29	3.42	3.67
337	3.15	0.930	0.953	0.954	2.81	2.87	3.01
720	2.18	0.755	0.749	0.897	1.91	1.91	2.06
94	2.17	0.747	0.761	0.950	1.86	1.90	2.01
1129	2.14	0.659	0.669	0.875	1.63	1.69	1.94
664	1.04	0.722	0.774	0.945	0.81	0.85	0.98

unit	raw d'	champ amp	no_norm amp	omission0 amp	champ d'	no_norm d'	omission0 d'
mean	4.497	0.834	0.849	0.939	4.151	4.200	4.368

The amplitude gain is **concentrated on the weakest units** — 1129 $0.659 \rightarrow 0.875$ (+0.22), 664 $0.722 \rightarrow 0.945$ (+0.22), 94 $0.747 \rightarrow 0.950$ (+0.20), 720 $0.755 \rightarrow 0.897$ (+0.14) — while the already-strong units (2143, 793) were near ceiling and barely move. This is the *opposite* of L2, whose amp gain landed on mid/strong units and left the weak ones smoothed (Section 5). So **omission0** fixes the undershoot **where it actually costs sorting** — the marginal units near the detection floor. Detection improves on nearly every unit, including the strongest (2143 $12.37 \rightarrow 12.57$).

The one caveat is visible in the same numbers: omission0 removes *less* white noise (SNR gain +1.20 vs the champion's +1.89), because seeing $t \pm 1$ lets a little temporally-correlated noise through. The offsetting evidence that this is a real spike gain, not leakage, is that **fixed-template d' rose by the same +0.204** — the denoised spikes project better onto the *true* template, which noise-copying could not produce. Still, this is a single seed-0 run; at $\sim 7\sigma$ it is very unlikely to be a lucky draw (unlike L2's +0.057), but it needs seed replication before it becomes the champion — and, given the size of the effect, it is the natural basis for a longer training run.

5. Per-unit amplitude: who gets smoothed, and why

The “0.85 amplitude” quoted everywhere is an *average*, and the average hides the interesting structure. Split per unit, a clean law appears: **how well a spike's peak survives denoising is set by how strong that unit already was**. The blind spot rebuilds the peak from its neighbours; for a loud, well-isolated unit those neighbours pin the peak down and it returns near full height, but for a faint unit the neighbours are mostly noise, so the estimator hedges toward baseline and the peak is smoothed away. This is textbook regression-to-the-mean, and it means the undershoot lives almost entirely on the marginal units that are hardest to sort in the first place.

amp_ratio is **strongly quality-dependent** — Spearman(amp, baseline d') = **+0.92**; vs peak size +0.88; vs SNR +0.79.

unit	baseline d'	champion amp	champ_l2 amp	no_norm amp	omission0 amp
2143	12.5	0.968	0.972	0.976	1.002
793	8.6	0.966	1.020	0.972	0.956
1143	5.3	0.881	0.918	0.897	0.946
1122	3.8	0.854	0.924	0.882	0.944
1300	4.1	0.857	0.879	0.859	0.923
720	2.2	0.755	0.735	0.749	0.897
94	2.2	0.747	0.754	0.761	0.950
664	1.0	0.722	0.744	0.774	0.945
1129	2.1	0.659	0.706	0.669	0.875

- Well-isolated units are preserved near-perfectly (~ 0.97); marginal low-SNR units lose 25–34%.
- Blind-spot reconstruction is a **shrinkage estimator**: weak spikes buried in noisy neighbours regress toward baseline.
- The “0.85 undershoot” is a **mean over a quality gradient**; the undershoot lives on units near the sorting floor (decoupled from detection).
- L2's amp gain concentrates on mid/high-quality units; low-SNR units stay undershot.
- **omission0 is the exception that proves the rule**: recovering the adjacent frames $t \pm 1$ lifts the *weak* units most (1129 $0.66 \rightarrow 0.88$, 664 $0.72 \rightarrow 0.95$, 94 $0.75 \rightarrow 0.95$, 720 $0.76 \rightarrow 0.90$), the one lever that undoes the shrinkage exactly where it costs sorting (Section 4.7). **no_norm** stays on the champion's gradient (null).

6. Conclusions

Pulling the threads together, the search fell into two phases. In the first — sweeping the loss function, capacity, output-head width, spike-weighting and SUPPORT branch wiring — almost every promising single-run gain evaporated once we measured the training-noise floor (Section 2): only capacity (base64) survived, as a small +0.06 d', with L2 a smaller free nudge. That made the ~ 0.3 d' detection/amplitude wall look fundamental. The second phase broke it. One temporal-design choice the champion had inherited for speed — hiding the immediately-adjacent frames $t \pm 1$ from the temporal branch (**omission=1**) — turned out to *be* the wall: simply not hiding them (**omission=0**) produced by far the largest gain in the study, on detection **and** amplitude at once. The yardstick in Section 2 was necessary to clear away the noise, but the real lever was an architectural assumption we had not questioned until we compared ourselves to SUPPORT.

Two decoupled axes:

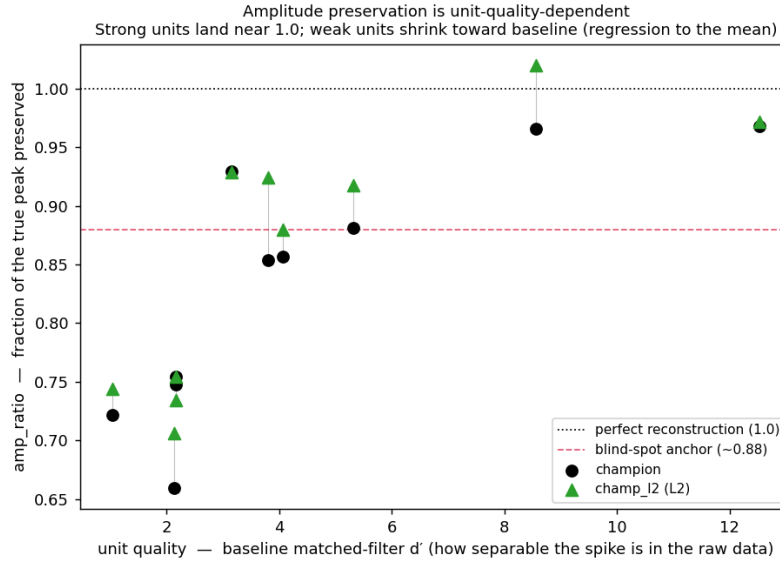


Figure 7 — amplitude preservation follows unit quality. Peak preserved (y) vs baseline separability (x). Strong units (right) ~ 1.0 ; weak units (left) 0.66–0.75. Green = L2, black = champion, grey links join the same unit; L2’s gains land on mid/strong units while the weakest stay smoothed.

1. **Detection (d'_{self}) — the sorting-relevant axis.** *Within the champion’s temporal design* (`omission=1`) it is remarkably flat: after replication the only reproducible mover is **capacity/base64** ($+0.063$, $p=0.019$), and L2 ($+0.057$, NS), spike-weighting, fuse-width, temporal_mult, multiscale and blind-spot width are all noise or harmful — every such model sits ~ 0.25 – 0.35 below raw (4.50). **But that wall is a property of the omission=1 design, not a fundamental limit:** `omission=0` clears $+0.204$ (~ 7 sigma), roughly halving the deep-vs-raw deficit (Section 4.7). So the reproducible detection levers, in order, are the **temporal design** (`omission`, $+0.20$) » **capacity** ($+0.06$); everything else is noise.
2. **Amplitude (amp_ratio) — invariant to capacity/loss/wiring (0.845 ± 0.01), but *not* to the temporal design.** Across capacity, fuse width, SUPPORT wiring and L2 it never moved more than $\sim +0.01$. The one lever that breaks it is the omission gap: **omission=0 lifts amp to 0.939** (Section 4.7) by handing the temporal branch the adjacent frames $t \pm 1$ — and it rescues exactly the weak marginal units the shrinkage smooths, the opposite of L2’s mid/strong-unit gains. So the undershoot was a property of the omission=1 design, not a hard limit.

Headroom ceiling (and why the surrogate mattered): spikes = 0.065% of samples $\rightarrow$ invisible in unweighted val loss $\rightarrow$ flat val floor $\rightarrow$ best-model step wanders 14–100% across seeds $\rightarrow$ large single-run sigma. Section 2.4 makes this concrete — reconstructing every spike *perfectly* would move the loss by only $\sim 4e-5$, $\sim 18x$ below its seed-to-seed noise. This is why replication is mandatory — and why the flat validation loss could never have surfaced `omission=0`; only the spike-level surrogate did.

Champion — current vs emerging. The *conservative* pick today is still **base32 + L2** — free (champion params and compute), never worse than the Charbonnier champion, though its replicated d' gain is small ($+0.057$, NS) and capacity is redundant with it. But the *emerging* champion is **omission=0** (optionally + L2): it moved detection **and** amplitude far more than anything else (d' 4.368, amp 0.939) at zero param cost, and it is the one change that broke the amplitude wall. Because it is a single seed, we are replicating and scaling it before promoting it (Section 7).

What would actually move the ceiling — and it did. The **champ_12** replicates walked the L2 headline back ($+0.13 \rightarrow +0.057$, NS) and no L2 combo stacked. The two architectural divergences from SUPPORT then landed very differently: dropping normalization (`no_norm`) does nothing, but **removing the omission gap (`omission=0`) is the largest single move in the whole sweep** — $+0.204$ d' (~ 7 sigma) *and* $+0.094$ amp, the first lever to break the amplitude invariance, concentrated on the marginal units that matter most (Section 4.7). This **reframes the search**: the detection/amplitude wall was substantially a consequence of the champion’s omission=1 blinding of $t \pm 1$ — exactly the SUPPORT temporal-design difference — not a loss or capacity limit. Because it is a single seed, we are now replicating and scaling it (Section 7) before promoting it to champion.

7. The omission gap at SUPPORT scale — and a direct test of undertraining

The sweep left two questions a single short run could not settle: does `omission=0` replicate, and were the models simply **undertrained**? Both are now answered by two matched runs trained $\sim 7\times$ longer (~ 3.3 M updates, ~ 17 h each), each writing 12 log-spaced checkpoints so amp and d' can be traced *against training progress* rather than read at a single endpoint (Figure 8).

(1) Does `omission=0` replicate? Its short-run $+0.204$ d' / $+0.094$ amp was a single seed-0 point — at ~ 7 sigma above the champion band very unlikely to be a draw (L2's $+0.057$ was ~ 2 sigma and did *not* hold), but the honest bar is a longer, independent run.

(2) Were the models undertrained? This is worth taking seriously, because the model whose design we are borrowing is trained *far* harder than anything in this study. SUPPORT (Eom et al., Nat. Methods 2023) fits a single recording for ~ 500 epochs with heavy augmentation — random crops plus the eight dihedral flips/rotations, redrawn every batch — on the order of **14 M gradient updates over ~ 2 days on one GPU**. Our entire sweep, by contrast, made a *single* pass over short slices with no augmentation — about **0.5 M updates in ~ 2 h**:

	our sweep	SUPPORT-scale runs (below)	SUPPORT (Eom 2023)
passes over data	1 epoch, 35 short slices	1 pass, one full ~ 2 h recording	~ 500 epochs, one movie
gradient updates	~ 0.5 M	~ 3.3 M	~ 14 M
augmentation	none	none	random crop + 8 dihedral
wall-clock	~ 2 h	~ 15 h	~ 2 days (1 GPU)

That is a $\sim 28\times$ gap in optimisation. And our earlier evidence that the model “converges in one epoch” was read off the **validation loss**, which is spike-blind (Section 2.4 measures how little spikes move it) — it cannot see whether the *spike-level* metrics keep improving past where the loss flattened. Section 2.4 also gives the mechanism: because the spike gradient is such a tiny fraction of each update, it accumulates slowly, so SUPPORT-like training lengths *might* be needed before spike quality saturates. A real possibility, then, was that part of the $0.25\text{--}0.35$ d' deficit was simply undertraining. The runs below test this directly — and note they still reach only $\sim 1/4$ of SUPPORT's update budget with no augmentation, so they are a *lower bound* on what still more training could buy.

The experiment. Two matched runs train on the **raw** 681532 ProbeC recording (the background of the very hybrid we score — no injected templates are ever seen, so there is no leakage) at **SUPPORT scale**: the full ~ 2 h recording streamed once through memory-bounded chunks, **~ 3.3 M updates ($\sim 7\times$ the sweep)**, `support_all` wiring, L2 loss, identical except for the single variable under test:

run	temporal design	rest
RUN1 (control)	<code>omission=1</code> , 3-frame hole (t+/-1 hidden)	<code>support_all</code> + L2, SUPPORT-scale
RUN2 (test)	<code>omission=0</code> , t+/-1 visible	<code>support_all</code> + L2, SUPPORT-scale

What the trajectories show (Figure 8). Three things, all clean:

The omission gap is real and survives long training. `omission=0` (RUN2) sits above `omission=1` (RUN1) at **every checkpoint from step 60 onward** — a steady **$+0.15\text{--}0.20$ d'** and **$+0.10$ amp** across the whole plateau, with a visibly sharper peak (fwhm ~ 0.98 vs $1.03\text{--}1.07$). The scale run does not merely replicate the short-run `omission=0` win, it slightly exceeds it (peak d' 4.44, amp 0.96). So the gap is a genuine architectural lever — not a seed artifact, not a training-amount effect.

Neither metric was undertrained — if anything the reverse. Both d' and amp **saturate within the first $\sim 1\text{--}10$ thousand updates** and then plateau, giving a little back, all the way to 3.3 M. d' peaks at step ~ 3.6 k (RUN1) / ~ 14 k (RUN2); amp peaks by $\sim 14\text{--}55$ k. Across the final *two decades* of training there is no gain to be had. Our short champion runs (~ 0.5 M updates) were therefore already **far past** saturation: the amp ~ 0.85 ceiling under `omission=1` is set by the omission gap, not by optimisation, and projecting SUPPORT's ~ 14 M updates onto these flat curves buys nothing for spike quality. The undertraining hypothesis is refuted.

The validation loss picked the wrong checkpoint (a Section 2.4 corollary). The val-MSE-selected `best_model` (gold stars) does not sit at the d'/amp optimum. RUN2's step-14 k checkpoint (d' 4.44 / amp 0.96) beats its own loss-selected best at step 2.73 M (4.40 / 0.95); RUN1's best-by-loss lands at step 231 k, long after its d' peaked near 3.6 k. Exactly as Section 2.4 predicts, whole-frame loss is a poor ruler for spike quality and best-checkpoint selection is close to a random draw along the plateau.

Resolving the three read-outs. Of the outcomes framed above: **RUN2 > RUN1 at every checkpoint** — yes, the omission gap holds at scale; **both climb past ~ 0.5 M** — no, both saturate two decades earlier. So the model genuinely

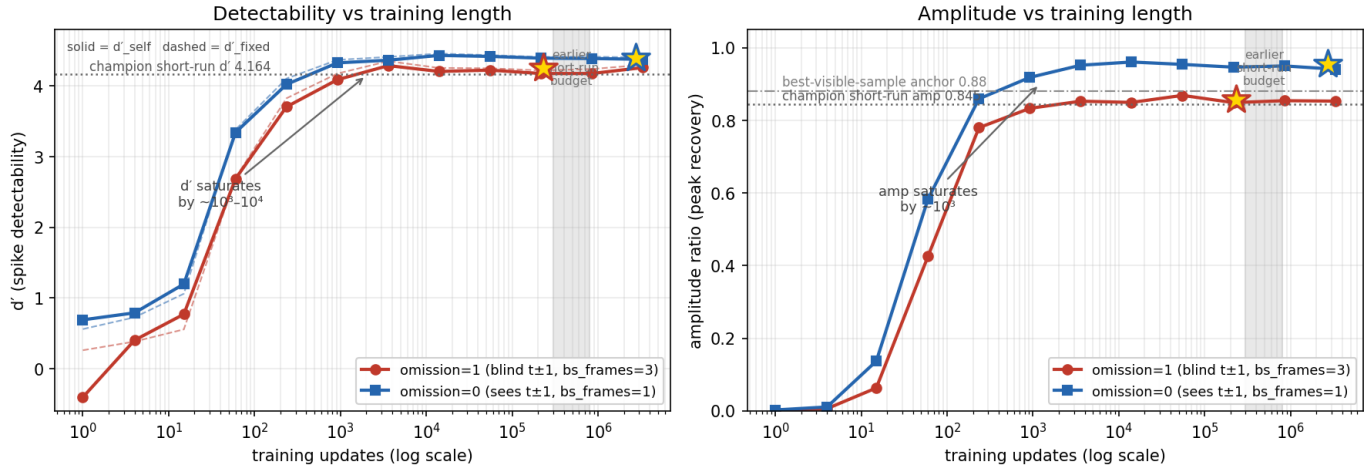


Figure 8 — SUPPORT-scale omission A/B (single-session 681532 ProbeC, ~3.3 M updates, seed 0). d' (left) and amplitude ratio (right) vs training updates on a log axis, for `omission=0` (blue) and `omission=1` (red); solid = d'_self , dashed = d'_fixed ; gold stars mark each run’s validation-loss-selected `best_model`. Both metrics saturate within the first $\sim 1\text{--}10$ thousand updates and are flat across the final two decades — the models are not undertrained — while `omission=0` holds a steady $+0.15\text{--}0.20$ d' / $+0.10$ amp lead with a sharper peak. Dotted lines: champion short-run means (d' 4.164, amp 0.845); dash-dot: the best-visible-sample amplitude anchor (0.88); grey band: the sweep’s ~ 0.5 M-update budget.

converges in a small fraction of one pass, and the short-run `omission=0` win (Section 4.7) is the whole story. The recommended champion becomes the current body **with** `omission=0` (optionally with L2); the amplitude/detectability ceiling this study chased is **architectural — the temporal design, not the training budget**.

One loose end this settles: RUN1’s mild *validation-loss* overfit (best at 231 k, then drifting up) is **not** leakage through its wider 3-frame hole — the blind-spot self-check reads exactly zero gradient from all three centre frames, and leakage would push the loss *down*, not up. It is the expected bias-variance signature: `omission=0`’s access to $t\pm 1$ gives it a near-universal local-interpolation solution (adjacent-sample correlation ~ 0.96) that acts as a strong implicit regulariser, whereas `omission=1`, blind to $t\pm 1$, must lean on higher-variance long-range and cross-channel structure that transfers less well from the training windows to the held-out one. The d' /amp trajectories show this costs almost nothing in spike terms — both peaked long before the loss turned.

Appendix A — Model glossary (one line each)

All models are the `fold` DeepInterpolation ephys denoiser (channels scattered onto the NP1 probe grid, W columns folded into the feature axis, a 1-D U-Net run along probe depth, fused per-channel with a probe-axis blind-spot branch over the centre frame). Params are absolute; loss-only variants share their body’s param count.

Baseline

- **champion / champ** (0.85M) — the reference denoiser: `base_channels=32`, `depth=3`, 3-frame temporal blind spot (`bs_frames=3`), `bs_channels=64`, `bs_depth=5`, `fuse_channels=64`, Charbonnier(`eps=0.4`) loss.

Loss shape

- **champ_12** (0.85M) — the champion with the training loss switched from Charbonnier to plain L2/MSE; nothing else changed. Highest single run (seed-0 d' 4.296) but a **lucky seed** — 4-seed mean 4.221 (± 0.057 , NS).
- **base64_12** (3.15M) — the same L2 loss on the double-width base64 body, to test whether loss and capacity stack (they don’t; 4.239).
- **champ_12_s1--3** (0.85M) — three extra training seeds of **champ_12** (with seed 0 $\rightarrow$ $n=4$), which walked its $+0.13$ d' back to a mean of 4.221 (± 0.057 , NS).
- **support_sd_12 / support_all_12 / fuse256_12** (0.85 / 0.90 / 0.93M) — L2 combined with the SUPPORT wiring (stage+dense, +multiscale) and the fat fuse head, to test whether anything stacks on L2. All land inside **champ_12**’s

seed spread (4.228–4.249); none clears the noise band.

Capacity

- **base64** (3.15M) — the champion with the U-Net base width doubled (base_channels 32→64).
- **arch** (12.59M) — a 15× enlargement (base_channels=64, depth=4, bs_channels=128, bs_depth=7) probing raw capacity; highest SNR, worst d’.
- **archL3** / **archL10** (12.59M) — the enlarged body plus spike-weighted loss at lambda=3 / lambda=10 (lambda10 over-amplifies peaks to amp 1.47).

Head / fuse-decoder width

- **fuse128** / **fuse256** / **fuse512** / **fuse1024** (0.87 / 0.93 / 1.14 / 1.97M) — the champion with the pointwise fusion head widened (fuse_channels 64→128...1024).
- **tmult8** (0.85M) — the champion with the temporal branch’s hand-off widened 8× (temporal_mult=8) instead of a single committed W-column prediction.
- **fuse256_tmult8** (0.93M) — the wide fuse head and the 8× temporal hand-off together.
- **fuse256_wL3** / **fuse512_wL3** (0.93 / 1.14M) — the wide-fuse variants combined with spike-weighted loss (lambda=3).

Spike-weighting (peak-aware loss; no param change)

- **weighted** (0.85M) — the champion with a magnitude spike-weight (loss weight = 1 + lambda; |neighbour amplitude|, lambda=3) driven by a centre-excluded neighbour spike detector.
- **l10g1** / **l10g2** (0.85M) — the magnitude weight pushed to lambda=10 with peakiness exponent gamma=1 / gamma=2.
- **champ_uL100**, **fuse256_uL30/100/300** (0.85 / 0.93M) — an *unbiased* saturating position-gate weight (up-weights a spike by ~lambda regardless of amplitude, common-median removed) at lambda=30–300.
- **fuse256_hard1000** (0.93M) — a hard binary spike gate (weight jumps only above a high threshold, lambda=1000, ~62% of the loss on spikes) — weighting’s best shot.

SUPPORT-style wiring

- **support_sd** (0.85M) — the champion’s blind-spot branch restored toward the SUPPORT design: dense re-injection of the centre input at every layer plus staging of the U-Net feature into the branch.
- **support_all** (0.90M) — **support_sd** plus a parallel multi-scale 5×5 hole-conv stack.

Blind-spot width

- **ho** (0.85M) — the champion with a 1-frame temporal blind spot (bs_frames=1) instead of 3.

Temporal / normalization design

- **omission0** (0.85M) — the champion with the temporal omission gap removed (omission=0, which also forces bs_frames=1): the temporal U-Net now sees the immediately-adjacent frames t+/-1 instead of hiding them, matching SUPPORT’s temporal design. **Largest single-run gain in the sweep** (amp 0.939, d’ 4.368) — but a single seed (Section 4.7).
- **no_norm** (0.85M) — the champion with GroupNorm removed from the temporal U-Net (SUPPORT ships without normalization). Essentially null (d’ 4.200, amp 0.849).

Seed replicates

- **champ_s1--4** / **fuse256_s1--4** / **base64_s1--4** — four extra training seeds each of champion / fuse256 / base64 (with seed 0 → n=5 per config), used only to measure the run-to-run training-noise floor.

best vs last — for every run, **best** = checkpoint at minimum validation loss, **last** = final training step; both were scored wherever they differ.

Appendix B — Per-unit amplitude across models

Every diagnostic run stores per-unit values, so amplitude preservation can be read model-by-model. Figure 9 and the table below give `amp_ratio` for each ground-truth unit (rows, sorted by baseline separability = unit quality) across a representative model set. **These are single-run (seed-0) values, so read the *columns* — the top-to-bottom quality gradient — not individual cells; per-unit model-to-model deltas are within run-to-run noise.**

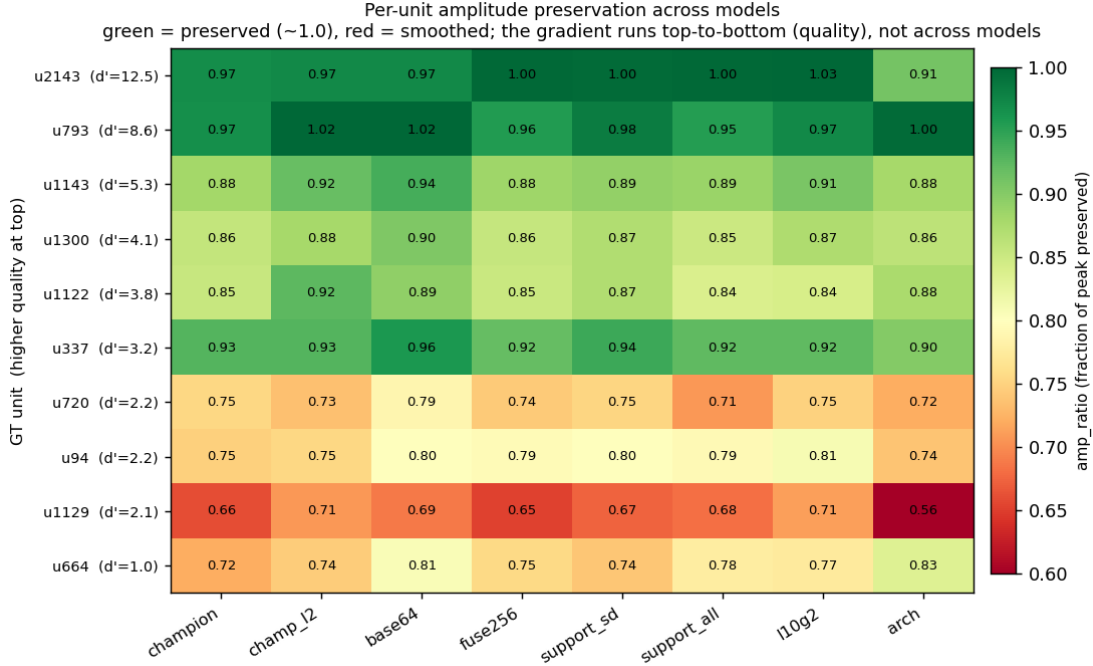


Figure 9 — per-unit amplitude preservation across models. `amp_ratio` for each GT unit (rows, highest quality at top) and model (columns); green = preserved (~1.0), red = smoothed. The gradient is vertical (unit quality) and essentially identical in every column — the undershoot is a property of the *unit*, not the architecture. The single reddest cell is `arch` on the weak unit 1129 (0.56).

unit	baseline d'	champion	champ_l2	base64	fuse256	support_sd	support_all	l10g2	arch	no_norm	omission0
2143	12.5	0.968	0.972	0.972	0.998	0.996	0.998	1.033	0.911	0.976	1.002
793	8.6	0.966	1.020	1.018	0.956	0.981	0.954	0.972	0.997	0.972	0.956
1143	5.3	0.881	0.918	0.936	0.883	0.892	0.887	0.907	0.879	0.897	0.946
1300	4.1	0.857	0.879	0.904	0.856	0.871	0.850	0.872	0.861	0.859	0.923
1122	3.8	0.854	0.924	0.894	0.852	0.872	0.839	0.841	0.876	0.882	0.944
337	3.2	0.930	0.929	0.959	0.919	0.941	0.923	0.923	0.901	0.953	0.954
720	2.2	0.755	0.735	0.788	0.743	0.752	0.708	0.749	0.721	0.749	0.897
94	2.2	0.747	0.754	0.801	0.794	0.801	0.793	0.808	0.739	0.761	0.950
1129	2.1	0.659	0.706	0.689	0.652	0.674	0.683	0.711	0.563	0.669	0.875
664	1.0	0.722	0.744	0.806	0.755	0.740	0.777	0.774	0.834	0.774	0.945
mean	—	0.834	0.858	0.877	0.841	0.852	0.841	0.859	0.828	0.849	0.939

The dominant pattern is vertical: strong units (top) are preserved near 1.0 in *every* model, weak units (bottom) are smoothed to 0.66–0.83 everywhere. Amplitude undershoot is a property of the *unit*, not the architecture — the same shrinkage picture as Section 5, now resolved model-by-model.

Appendix C — Per-unit detection (d') across models

The detection metric can be read per unit the same way. Because absolute d' is dominated by each unit's intrinsic separability (raw d' spans 1.0–12.5), the informative view is the **change** from denoising, d'_deep - d'_raw (Figure 10): a negative value means denoising made that unit *harder* to detect.

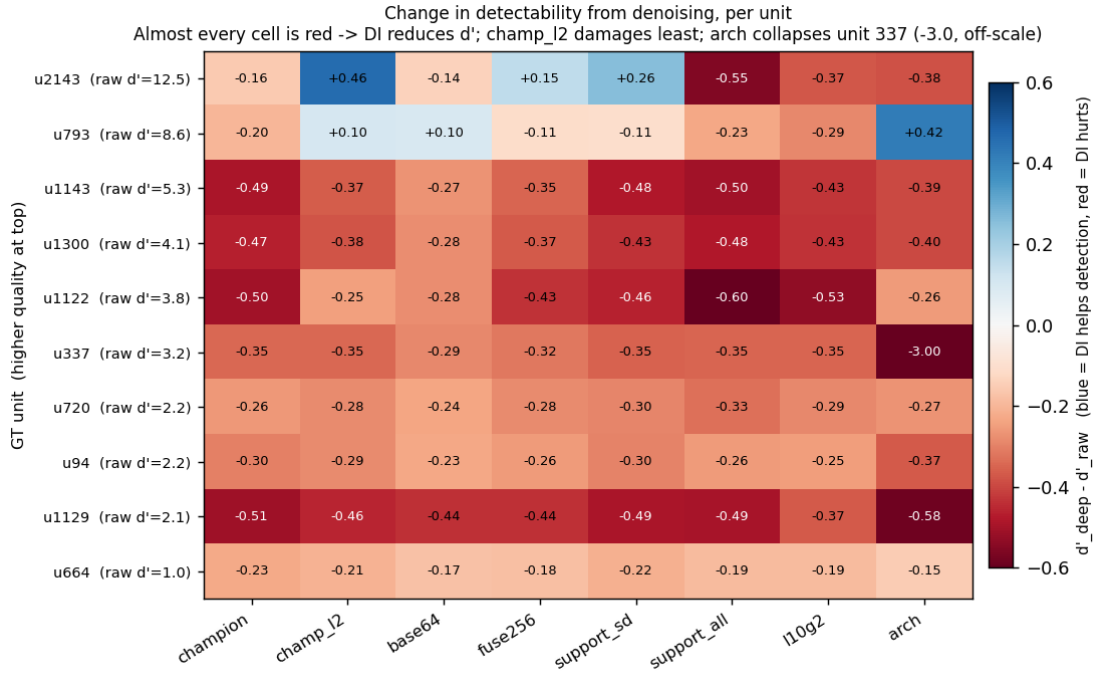


Figure 10 — change in detectability from denoising, per unit. d'_deep - d'_raw for each GT unit (rows, best at top) and model (columns); red = DI *hurts* detection, blue = DI helps. **Almost every cell is red** — denoising reduces separability for nearly every unit (the cost that motivates the whole search). **champ_l2** damages least (and turns the two strongest units positive); **arch** is erratic and **catastrophically collapses unit 337 (-3.0, off the colour scale): d' 3.15 → 0.16.**

Absolute d'_self per unit (raw = the undenoised baseline; every model mean is below it):

unit	raw d'	champion	champ_l2	base64	fuse256	support_sd	support_all	l10g2	arch	no_norm	omission0
2143	12.53	12.37	12.99	12.39	12.68	12.79	11.98	12.16	12.15	12.28	12.57
793	8.57	8.37	8.67	8.66	8.45	8.46	8.34	8.28	8.98	8.47	8.43
1143	5.32	4.84	4.96	5.05	4.97	4.84	4.82	4.90	4.93	4.95	5.14
1300	4.07	3.61	3.70	3.79	3.71	3.64	3.60	3.64	3.68	3.65	3.88
1122	3.80	3.29	3.55	3.52	3.36	3.34	3.20	3.26	3.54	3.42	3.67
337	3.15	2.81	2.81	2.86	2.83	2.80	2.80	2.80	0.16	2.87	3.01
720	2.18	1.91	1.89	1.94	1.89	1.88	1.85	1.89	1.90	1.91	2.06
94	2.17	1.86	1.88	1.94	1.91	1.87	1.91	1.92	1.80	1.90	2.01
1129	2.14	1.63	1.69	1.70	1.71	1.66	1.65	1.78	1.56	1.69	1.94
664	1.04	0.81	0.83	0.87	0.86	0.82	0.85	0.85	0.89	0.85	0.98
mean	4.497	4.151	4.296	4.273	4.238	4.209	4.099	4.148	3.958	4.200	4.368

Two things stand out: every column mean sits below the raw 4.497 (denoising costs separability on average — the core problem), and the single catastrophic cell (**arch** erasing unit 337) shows how added capacity can fail unpredictably on individual units, dragging its mean to the bottom of the sweep.